VSTATD-IF: `vstatd` interface specification

Document	VSTATD-IF, revision 1.3
Product	`vstatd`, vehicle status service on the CC-1 cockpit computer
Status	Released for the current `vstatd` release
Related	CC-1 platform notes (CC1-PN), CAN database `cc1-gateway.dbc`, signal mapping `vss-mapping.yaml`

> All names, values and behaviour in this document are synthetic and were written for the AI-DLC embedded sample. They describe no real vehicle, supplier or product.

§0 About this document

§0.1 Scope. This document specifies what `vstatd` reads from the CAN gateway, how it decodes and supervises the signals, what it offers to cockpit apps, how it reports errors and what it writes to its runtime log. It is the reference for any change to `vstatd`: a change that alters behaviour described here needs an approved specification change first.

§0.2 How to cite. Cite this document as `[VSTATD-IF p.<page> §<section>]`, for example `[VSTATD-IF p.7 §4.1]`. Each page ends with its page number; the rendered PDF has the same page breaks.

§0.3 Normative words. *Shall* is a requirement. *Should* is a recommendation that a change may depart from only with a recorded reason. *May* is a permission.

§0.4 Signal names. Signal paths use the names of the COVESA Vehicle Signal Specification (VSS), release v6.1, so that apps and tools can rely on a public catalogue. `vstatd` uses the names, data types, units and allowed values only; it does not implement the VSS data model or a VSS server. Several of these signals are VSS actuators; `vstatd` publishes their current value only and never sets a target value (§1.3).

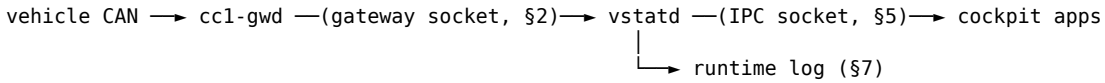
§0.5 Revision history.

Revision	Change
1.3	Tyre pressures for all four wheels; exterior temperature moved to its own message (0x401)
1.2	Start-up grace period added to message supervision (§4.2)
1.1	Subscriber queue limit and overflow event (§5.4, §7.1)
1.0	First release

§1 Overview

§1.1 Role. `vstatd` keeps the latest value of a fixed set of vehicle signals and serves them to cockpit apps (instrument cluster, HMI, logging). It reads CAN frames from the CAN gateway daemon `cc1-gwd`, decodes them with the CAN database `cc1-gateway.dbc`, supervises each message for timeouts and publishes each signal under its VSS path.

§1.2 Context.



§1.3 What `vstatd` does not do. It does not send CAN frames, does not write to any vehicle function and does not store history: an app that needs history keeps it itself. It does not interpret the meaning of a signal beyond the decoding and range rules in §3.

§1.4 Terms.

Term	Meaning
Frame	One classic CAN frame (identifier, 0 to 8 data bytes) as forwarded by `cc1-gwd`
Message	A CAN frame layout in `cc1-gateway.dbc`, identified by its CAN ID
Raw value	The integer read from a frame's bits, before scaling
Physical value	The raw value after scaling (§3.3), in the unit of §3.1
SNA	Signal not available: a raw value the sender uses to say it has no valid value
Status	`AVAILABLE`, `UNAVAILABLE` or `INVALID` (§3.4)
Client	A cockpit app connected to the IPC socket

§2 Gateway socket

§2.1 Connection. `vstatd` shall connect to the Unix stream socket `/run/cc1-gwd/can.sock` as a client. The gateway sends; `vstatd` never writes to this socket.

§2.2 Records. The gateway sends text records, one per line, ended by `\n`. Fields are separated by one space.

Record	Format	Example
Frame	`F <t_us> <can_id> <dlc> <data>`	`F 1532000 101 8 C409000000000000`
Heartbeat	`H <t_us>`	`H 1600000`

- `t_us`: gateway receive time in microseconds since the gateway started, decimal.
- `can_id`: CAN identifier in hexadecimal without prefix, 11-bit (`0` to `7FF`).
- `dlc`: number of data bytes, decimal, `0` to `8`.
- `data`: exactly `2 × dlc` hexadecimal digits, byte 0 first.
- Hexadecimal digits may be upper or lower case.

§2.3 Byte and bit order. Signals are little-endian (Intel byte order). Bit 0 is the least significant bit of byte 0. Start bits and lengths are those of `cc1-gateway.dbc`.

§2.4 Heartbeat. The gateway sends a heartbeat record every 100 ms whether or not frames arrive.

§2.5 Records that `vstatd` cannot use.

- A record that does not match §2.2 shall be discarded. `vstatd` shall log `GATEWAY_RECORD_INVALID` (§7.1), at most once per 10 s.
- A frame whose CAN ID is not in §3.2 shall be ignored without a log entry.
- A frame whose `dlc` is less than 8 for a message of §3.2 shall be discarded and logged as in the first bullet. The signals of that message keep their value and status; the frame does not count as received for §4.1.

§3 Signals

§3.1 Signal table, part A. The signals that `vstatd` publishes, with the CAN signal each is decoded from. The bit positions, lengths, factors and offsets in `cc1-gateway.dbc` are normative; this table repeats them for review.

VSS signal (`vstatd` path)	CAN message.signal	Raw to physical	Valid range	SNA raw	Log deadband
`Vehicle.LowVoltageSystemState`	0x100 VCU_PowerState.LvSystemState	3 bit enum, see §3.5	raw 0–5	7	any change
`Vehicle.LowVoltageBattery.CurrentVoltage`	0x100 VCU_PowerState.LvBatteryVoltage	10 bit unsigned, raw × 0.02	6 to 18 V	1023	0.1 V (raw 5)
`Vehicle.Speed`	0x101 VCU_Speed.VehicleSpeed	16 bit unsigned, raw × 0.01	0 to 300 km/h	65535	0.5 km/h (raw 50)
`Vehicle.Powertrain.Transmission.CurrentGear`	0x102 VCU_Gear.CurrentGear	8 bit signed, value as is	−1 to 8	−128	any change
`Vehicle.Powertrain.Transmission.SelectedGear`	0x102 VCU_Gear.SelectedGear	8 bit signed, see §3.5	−1, 0, 126, 127	−128	any change
`Vehicle.Chassis.ParkingBrake.IsEngaged`	0x102 VCU_Gear.ParkingBrakeEngaged	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.TraveledDistance`	0x103 VCU_Odometer.TraveledDistance	24 bit unsigned, raw × 100	0 to 1677721400 m	16777215	100 m (raw 1)
`Vehicle.Powertrain.TractionBattery.StateOfCharge.Displayed`	0x200 BMS_Status.SocDisplayed	8 bit unsigned, raw × 0.5	0 to 100 percent	255	0.5 percent (raw 1)
`Vehicle.Powertrain.Range`	0x200 BMS_Status.RemainingRange	16 bit unsigned, raw × 10	0 to 655340 m	65535	1000 m (raw 100)
`Vehicle.Powertrain.TractionBattery.Charging.IsCharging`	0x200 BMS_Status.IsCharging	2 bit: 0 false, 1 true, 2 error	false, true	3	any change

§3.1 Signal table, part B.

VSS signal (`vstatd` path)	CAN message.signal	Raw to physical	Valid range	SNA raw	Log deadband
`Vehicle.Body.Lights.Beam.Low.IsOn`	0x300 BCM_Lights.LowBeamOn	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Body.Lights.Beam.High.IsOn`	0x300 BCM_Lights.HighBeamOn	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Body.Lights.DirectionIndicator.Left.IsSignaling`	0x300 BCM_Lights.IndicatorLeft	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Body.Lights.DirectionIndicator.Right.IsSignaling`	0x300 BCM_Lights.IndicatorRight	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Body.Lights.Hazard.IsSignaling`	0x300 BCM_Lights.HazardOn	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Cabin.Door.Row1.DriverSide.IsOpen`	0x301 BCM_Closures.DriverDoorOpen	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Cabin.Door.Row1.PassengerSide.IsOpen`	0x301 BCM_Closures.PassengerDoorOpen	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Body.Trunk.Rear.IsOpen`	0x301 BCM_Closures.TrunkOpen	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Cabin.Seat.Row1.DriverSide.IsBelted`	0x301 BCM_Closures.DriverBelted	2 bit: 0 false, 1 true, 2 error	false, true	3	any change
`Vehicle.Chassis.Axle.Row1.Wheel.Left.Tire.Pressure`	0x400 TPMS_Pressures.TirePressureFL	8 bit unsigned, raw × 2	0 to 500 kPa	255	4 kPa (raw 2)
`Vehicle.Chassis.Axle.Row1.Wheel.Right.Tire.Pressure`	0x400 TPMS_Pressures.TirePressureFR	8 bit unsigned, raw × 2	0 to 500 kPa	255	4 kPa (raw 2)
`Vehicle.Chassis.Axle.Row2.Wheel.Left.Tire.Pressure`	0x400 TPMS_Pressures.TirePressureRL	8 bit unsigned, raw × 2	0 to 500 kPa	255	4 kPa (raw 2)
`Vehicle.Chassis.Axle.Row2.Wheel.Right.Tire.Pressure`	0x400 TPMS_Pressures.TirePressureRR	8 bit unsigned, raw × 2	0 to 500 kPa	255	4 kPa (raw 2)
`Vehicle.Exterior.AirTemperature`	0x401 BCM_Exterior.ExteriorAirTemp	8 bit unsigned, raw × 0.5 − 40	−40 to 85 Celsius	255	0.5 Celsius (raw 1)

§3.2 Messages, cycle times and timeouts. The cycle time is the sender's nominal period (DBC attribute `GenMsgCycleTime`). The timeout is used by §4.1.

CAN ID	Message	Sender	Cycle	Timeout (3 × cycle)
0x100	VCU_PowerState	VCU	100 ms	300 ms
0x101	VCU_Speed	VCU	20 ms	60 ms
0x102	VCU_Gear	VCU	50 ms	150 ms
0x103	VCU_Odometer	VCU	1000 ms	3000 ms
0x200	BMS_Status	BMS	100 ms	300 ms
0x300	BCM_Lights	BCM	100 ms	300 ms
0x301	BCM_Closures	BCM	100 ms	300 ms
0x400	TPMS_Pressures	TPMS	1000 ms	3000 ms
0x401	BCM_Exterior	BCM	1000 ms	3000 ms

§3.3 Decoding and scaling.

- The physical value shall be `raw × factor + offset`, with the factor and offset of §3.1.
- Raw values are at most 24 bits long (§3.1). For the signals with an integer factor, `raw × factor` therefore stays below 2^{32}, so unsigned 32-bit arithmetic cannot overflow; the largest product is 16 777 214 × 100 = 1 677 721 400 for `Vehicle.TraveledDistance`.
- Signals with a fractional factor shall be computed in double precision and published rounded to the decimals of §5.3.
- A raw value equal to the signal's SNA raw value shall not be scaled (see §3.4).

§3.4 Status. Every signal has a status, published with its value.

Status	When	Value published
`UNAVAILABLE`	Before the first frame of its message, after a timeout (§4.1) or gateway loss (§4.3), or when the raw value is the SNA value	None (`-`)
`INVALID`	The raw value is not SNA but the physical value is outside the valid range of §3.1 (for `SelectedGear`, not one of its four values), or a two-bit flag reads 2 (error)	None (`-`)
`AVAILABLE`	The physical value is inside the valid range	The physical value

- Out-of-range values shall not be clamped to the range.
- An `INVALID` signal shall be logged as `SIGNAL_INVALID` (§7.1), at most once per 10 s per signal.

§3.5 Enumerations and flags.

- `Vehicle.LowVoltageSystemState` (3 bits): 0 `UNDEFINED`, 1 `LOCK`, 2 `OFF`, 3 `ACC`, 4 `ON`, 5 `START`; 6 is an error (`INVALID`); 7 is SNA. The value is published as the VSS string.
- `Vehicle.Powertrain.Transmission.SelectedGear` (signed 8 bits): −1 reverse, 0 neutral, 126 park, 127 drive, using the VSS codes. VSS also defines codes for single forward gears; this vehicle does not send them. Any other value except the SNA value −128 is `INVALID`.
- `Vehicle.Powertrain.Transmission.CurrentGear` (signed 8 bits): −1 reverse, 0 neutral, 1 to 8 forward gears.
- Two-bit flags: 0 `false`, 1 `true`, 2 error, 3 SNA.

§4 Supervision

§4.1 Message timeout. For each message of §3.2, `vstatd` shall record the time it last received a usable frame (§2.5). When no usable frame has arrived for longer than the message's timeout, all signals of the message shall become `UNAVAILABLE`. `vstatd` shall log `SIGNAL_TIMEOUT` once for the message, not once per signal, and notify the subscribers of each signal (§5.3). The next usable frame ends the timeout: the signals are decoded again, and `SIGNAL_RECOVERED` is logged once for the message. Timeouts shall be detected within 10 ms of their expiry.

§4.2 Start-up grace period. Supervision of a message shall start at its first usable frame, or 2.0 s after `GATEWAY_CONNECTED`, whichever is earlier. When supervision starts without a usable frame, the timeout is counted from the start of supervision. Until then a message that has not been received is not in timeout and produces no `SIGNAL_TIMEOUT`; its signals are `UNAVAILABLE` because they have no value yet (§3.4). The grace period applies after every `GATEWAY_CONNECTED`, including reconnections (§4.4).

§4.3 Gateway loss. When no line of any kind, usable or not (§2.2), has arrived for 500 ms, or the socket is closed or returns an error, `vstatd` shall log `GATEWAY_LOST` once, set every signal to `UNAVAILABLE`, notify subscribers, and close the socket. Message timeouts are not logged separately while the gateway is lost.

§4.4 Reconnection. After `GATEWAY_LOST`, or when the first connection attempt fails, `vstatd` shall try again after 100 ms, then double the delay after each failed attempt up to 3 200 ms, and keep trying at 3 200 ms. A successful connection resets the delay to 100 ms and is logged as `GATEWAY_CONNECTED`. Connection attempts that fail shall not be logged individually; `GATEWAY_RETRY` is logged once per outage, when the delay first reaches 3 200 ms.

§4.5 Time base. All supervision times use the monotonic clock of CC-1, not the gateway's `t_us` and not wall time.

§5 IPC interface for apps

§5.1 Socket. `vstatd` shall listen on the Unix stream socket `/run/vstatd/vstatd.sock`, mode `0660`, group `vstat`. The protocol is text, one request or response per line, ended by `\n`, at most 256 bytes including the newline.

§5.2 Requests.

Request	Meaning
`GET <path>`	Return the current value and status of one signal
`SUB <path> [<interval_ms>]`	Subscribe to a signal; updates are sent at most every `interval_ms` (default 100, range 50 to 10 000)
`UNSUB <path>`	End a subscription
`LIST`	Return every published path, one `PATH <path>` line each in the order of §3.1, followed by `OK`

`SUB` on a path that is already subscribed replaces its interval. `UNSUB` of a path that is not subscribed answers `OK`.

§5.3 Responses.

- `VAL <path> <value> <status> <t_ms>` answers `GET` and carries every subscription update. `value` is `-` unless the status is `AVAILABLE`. Numbers use a dot as decimal separator, with two decimals for `Vehicle.Speed` and `Vehicle.LowVoltageBattery.CurrentVoltage`, one for state of charge and exterior temperature, and none otherwise. Flags are `true` or `false`. `t_ms` is the time of the decoding step (or of the timeout) on the CC-1 monotonic clock since boot, in milliseconds; the runtime log counts from process start instead (CC1-PN §5.2).
- `OK` confirms `SUB`, `UNSUB` and the end of `LIST`. After the `OK` of a `SUB`, one `VAL` with the current value follows at once. `ERR <code> <text>` reports an error (§6.1).
- A subscription update shall be sent when the value or the status changes, but not more often than the subscription's interval. When updates are held back by the interval, the client receives the latest value at the end of the interval, not every intermediate value. A status change to `UNAVAILABLE` shall be sent at once.

§5.4 Limits.

- At most 16 clients at a time. At most 32 subscriptions per client.
- Each client has an outgoing queue of 256 lines. When it is full, the oldest line is dropped and `CLIENT_QUEUE_OVERFLOW` is logged, at most once per second per client. A slow client shall never block `vstatd`.
- Clients that send nothing are not disconnected.

§6 Error behaviour

§6.1 Error codes.

Code	Condition	Effect
`E01`	Unknown request word	`ERR E01 unknown request`; connection stays open
`E02`	Path not in §3.1	`ERR E02 unknown path`; connection stays open
`E03`	Line longer than 256 bytes	`ERR E03 line too long`; the line is discarded up to the next newline
`E04`	33rd subscription of a client	`ERR E04 too many subscriptions`; existing subscriptions unchanged
`E05`	Interval outside 50 to 10 000 ms	`ERR E05 interval out of range`; no subscription made
`E06`	17th client	`ERR E06 too many clients`, then the connection is closed

§6.2 Client errors. Every `ERR` response shall be logged as `REQUEST_ERROR` with the code, at most once per second per client. A client error shall never affect other clients or the gateway side.

§6.3 Gateway errors. Gateway records that cannot be used are handled by §2.5 and gateway loss by §4.3. Neither shall stop `vstatd`.

§6.4 Fatal errors. `vstatd` shall exit with a non-zero status only when it cannot create its IPC socket at start-up or cannot allocate memory. It shall log `SERVICE_STOP` with the reason before exiting. Any other error is handled and logged without exiting.

§6.5 Repeated logs. The rate limits in this document apply per signal, per client or per message as stated. An entry is written when no entry of the same kind was written within the rate limit; "same kind" is the event with its signal, client or reason. A log entry suppressed by a rate limit is counted, and the next entry of the same kind carries the count of suppressed entries in its `suppressed` field (§7.1).

§7 Runtime log

§7.1 Events. The line format is defined in the CC-1 platform notes (CC1-PN §5). The events are:

Event	Fields besides time and event	When
SERVICE_START	version	Process started
SERVICE_READY	—	IPC socket listening (§8.1)
GATEWAY_CONNECTED	—	Gateway socket connected (§4.4)
GATEWAY_LOST	reason	§4.3
GATEWAY_RETRY	delay_ms	§4.4
GATEWAY_RECORD_INVALID	reason, suppressed	§2.5
SIGNAL_VALUE	path, value, status	§7.2
SIGNAL_TIMEOUT	message	§4.1
SIGNAL_RECOVERED	message	§4.1
SIGNAL_INVALID	path, raw, suppressed	§3.4
CLIENT_CONNECTED / CLIENT_DISCONNECTED	client	Client connects or closes
CLIENT_REJECTED	client	E06
CLIENT_QUEUE_OVERFLOW	client, dropped	§5.4
REQUEST_ERROR	client, code, suppressed	§6.2
SERVICE_STOP	reason	Normal stop or §6.4

§7.2 Value logging. SIGNAL_VALUE shall be logged when a signal's status changes, and when its raw value differs from the raw value last logged by at least the raw log deadband of §3.1 (the deadband divided by the factor, an integer for every signal). For signals with "any change", every change of value is logged. Values use the formatting of §5.3. Subscriptions do not affect value logging.

§7.3 Order. Log entries of one decoding step shall be written in this order: SIGNAL_RECOVERED, then the SIGNAL_VALUE entries of the message in the order of §3.1. A timeout writes SIGNAL_TIMEOUT, then the SIGNAL_VALUE entries of the message in the order of §3.1; messages that time out in the same step follow the order of §3.2. GATEWAY_LOST comes before the SIGNAL_VALUE entries it causes.

§7.4 Field values. reason of GATEWAY_LOST: silence, closed or error. reason of GATEWAY_RECORD_INVALID: format or short_dlc. reason of SERVICE_STOP: signal, ipc_socket or out_of_memory. message: the message name of §3.2. client: a decimal number, 1 for the first client accepted since process start, counting up and never reused. raw: decimal, signed for signed signals. suppressed and dropped: decimal counts since the previous entry of the same kind, 0 when none. delay_ms: decimal.

§8 Start-up, stop and resources

§8.1 Start-up. `vstatd` shall log `SERVICE_START`, create the IPC socket, report readiness to the service manager and log `SERVICE_READY` within 1.5 s of process start. It shall not wait for the gateway to become ready: apps may connect while every signal is still `UNAVAILABLE`. The gateway connection (§4.4) starts after `SERVICE_READY`.

§8.2 Stop. On `SIGTERM`, `vstatd` shall stop accepting clients, close all connections, log `SERVICE_STOP` with reason `signal` and exit with status 0 within 500 ms.

§8.3 Resources. The memory, CPU and binary-size limits of `vstatd` are set in the CC-1 platform notes (CC1-PN §3). `vstatd` shall not grow memory with the number of frames received; its memory depends only on the number of clients, subscriptions and queued lines, all bounded by §5.4.

§8.4 Threads. The specification does not prescribe a threading model. Any shared state between threads shall be protected so that a client never receives a value and status from different decoding steps.